



Lecture 2

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

July 21, 2026

History

Two major ways of implementing Programming Languages

History

Two major ways of implementing Programming Languages

- Interpreter

History

Two major ways of implementing Programming Languages

- Interpreter
- Compiler

History

Two major ways of implementing Programming Languages

- Interpreter
- Compiler

Early Machines

- IBM developed 704 in 1954. All programming was done in assembly language. The cost of software development far exceeded the cost of hardware. Low productivity.

History

Two major ways of implementing Programming Languages

- Interpreter
- Compiler

Early Machines

- IBM developed 704 in 1954. All programming was done in assembly language. The cost of software development far exceeded the cost of hardware. Low productivity.
- Speedcoding interpreter: programs ran about 10 times slower than hand written assembly code

History

Two major ways of implementing Programming Languages

- Interpreter
- Compiler

Early Machines

- IBM developed 704 in 1954. All programming was done in assembly language. The cost of software development far exceeded the cost of hardware. Low productivity.
- Speedcoding interpreter: programs ran about 10 times slower than hand written assembly code
- John Backus (in 1954): Proposed a program that translated high level expressions into native machine code. Skepticism all around. Most people thought it was impossible

History

Two major ways of implementing Programming Languages

- Interpreter
- Compiler

Early Machines

- IBM developed 704 in 1954. All programming was done in assembly language. The cost of software development far exceeded the cost of hardware. Low productivity.
- Speedcoding interpreter: programs ran about 10 times slower than hand written assembly code
- John Backus (in 1954): Proposed a program that translated high level expressions into native machine code. Skepticism all around. Most people thought it was impossible
- Fortran I project (1954-1957): The first compiler was released

Fortran 1

- The whole new field of compiler design was started

Fortran 1

- The whole new field of compiler design was started
- More than half the programmers were using Fortran by 1958

Fortran 1

- The whole new field of compiler design was started
- More than half the programmers were using Fortran by 1958
- The development time was cut down to half

Fortran 1

- The whole new field of compiler design was started
- More than half the programmers were using Fortran by 1958
- The development time was cut down to half
- Led to the enormous amount of theoretical work (lexical analysis, parsing, optimization, structured programming, code generation, error recovery, etc.)

Fortran 1

- The whole new field of compiler design was started
- More than half the programmers were using Fortran by 1958
- The development time was cut down to half
- Led to the enormous amount of theoretical work (lexical analysis, parsing, optimization, structured programming, code generation, error recovery, etc.)
- **Modern compilers preserve the basic structure of the Fortran I compiler !!!**

What are Compilers

What are Compilers

Basics

What are Compilers

Basics

- What is High-Level Languages
 - ▶ Expressive: matches our notion of languages (and application?!)

What are Compilers

Basics

- What is High-Level Languages
 - ▶ Expressive: matches our notion of languages (and application?!)
 - ▶ Redundant to help avoid programming errors

What are Compilers

Basics

- What is High-Level Languages
 - Expressive: matches our notion of languages (and application?!)
 - Redundant to help avoid programming errors
- Can the computer understand this?

What are Compilers

Basics

- What is High-Level Languages
 - ▶ Expressive: matches our notion of languages (and application?!)
 - ▶ Redundant to help avoid programming errors
- Can the computer understand this?
- Who will convert it?

What are Compilers

Basics

- What is High-Level Languages
 - ▶ Expressive: matches our notion of languages (and application?!)
 - ▶ Redundant to help avoid programming errors
- Can the computer understand this?
- Who will convert it?
- Goals of translation

What are Compilers

Basics

- What is High-Level Languages
 - ▶ Expressive: matches our notion of languages (and application?!)
 - ▶ Redundant to help avoid programming errors
- Can the computer understand this?
- Who will convert it?
- Goals of translation
 - ▶ Good performance for the generated code

What are Compilers

Basics

- What is High-Level Languages
 - ▶ Expressive: matches our notion of languages (and application?!)
 - ▶ Redundant to help avoid programming errors
- Can the computer understand this?
- Who will convert it?
- Goals of translation
 - ▶ Good performance for the generated code
 - ▶ Good compile-time performance

What are Compilers

Basics

- What is High-Level Languages
 - ▶ Expressive: matches our notion of languages (and application?!)
 - ▶ Redundant to help avoid programming errors
- Can the computer understand this?
- Who will convert it?
- Goals of translation
 - ▶ Good performance for the generated code
 - ▶ Good compile-time performance
 - ▶ Maintainable code

What are Compilers

Basics

- What is High-Level Languages
 - ▶ Expressive: matches our notion of languages (and application?!)
 - ▶ Redundant to help avoid programming errors
- Can the computer understand this?
- Who will convert it?
- Goals of translation
 - ▶ Good performance for the generated code
 - ▶ Good compile-time performance
 - ▶ Maintainable code
 - ▶ What about correctness?